

P4032R1

Strong ordering for `meta::info`

Published Proposal, 2026-03-24

Author:

[Lénárd Szolnoki](#)

Audience:

SG17, EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

1 Revision history

1.1 R1

2 Proposal

3 Background

4 Motivation

5 Implementation

6 Wording

6.1 [basic.fundamental]

6.2 [expr.spaceship]

6.3 [expr.rel]

6.4 [over.built]

6.5 [compare.type]

6.6 [cpp.predefined]

References

Informative References

§ 1. Revision history

§ 1.1. R1

- Not forwarding type aliases directly to `std::type_order` in [implementation](#).
- Using `std::type_order_v` instead of `std::type_order` for comparison table in [motivation](#) and in [implementation](#).
- Fixed small mistake in [background](#) code snippet.

§ 2. Proposal

I propose `meta::info` to be three way comparable with an implementation-defined strong order, consistent with `std::type_order` for reflections that represent types.

§ 3. Background

[P2830R10] introduced `type_order`, which exposes an implementation-defined `constexpr` strong order on types.

[P2996R13] (Reflecton for C++26) introduced `meta::info` as a structural type. This means that class template specializations can have constant template arguments of `meta::info` type. Therefore arbitrary reflection values are subject to ordering through indirection to class template specializations:

```
template <std::meta::info> struct S {};  
  
// what comes first? int or the global namespace?  
constexpr std::strong_order b = std::type_order<S<^int>, S<^::>>::value;
```

§ 4. Motivation

Being able to compare `meta::info` directly makes metaprogramming that needs to sort types, functions, etc... into some canonical order with standard algorithms more convenient.

Consider `type_set`, one of the motivating examples of [P2830R10]:

With <code>std::type_order</code> (status quo)	With <
<pre>struct type_less { constexpr bool operator()(std::meta::info lhs, std::meta::info rhs) const noexcept { auto ordering_info = substitute(^type_order_v, { substitute(^S, {std::meta::reflect_constant(lhs)}), substitute(^S, {std::meta::reflect_constant(rhs)}), }); return extract<const std::strong_ordering&>(ordering_info) < 0; } }; template <typename ...> struct type_set_impl; template <typename ...Ts> using type_set = [:substitute(^type_set_impl, std::set<meta::info, type_less>{^Ts...})];</pre>	<pre>template <typename ...> struct type_set_impl; template <typename ...Ts> using type_set = [:substitute(^type_set_impl, std::set<meta::info, type_less>{^Ts...})];</pre>

Moreover an ordering over reflections generalizes to cases where we want to order something other than types. For example getting a canonical order of annotations over a particular entity, instead of the lexical order.

§ 5. Implementation

[P2830R10] argues that any `operator<=>(meta::info, meta::info)` should be consistent with `type_order`. I agree.

Implementing such a comparison between arbitrary `meta::info` values is possible, based on an existing `type_order` implementation:

```
template <std::meta::info> struct _Helper {};  
  
constexpr std::strong_ordering compare(std::meta::info a, std::meta::info b) {
```

```
constexpr auto is_exactly_type = [](std::meta::info i){
    return is_type(i) && !is_type_alias(i);
};
if (is_exactly_type(a) && is_exactly_type(b)) {
    // ensure that on types meta::info ordering is consistent with type_order
    auto ordering_info = substitute(
        ^^std::type_order_v, {a, b}
    );
    return extract<const std::strong_ordering&>(ordering_info);
} else if (!is_exactly_type(a) && !is_exactly_type(b)) {
    // indirect through helper class template for non-type reflections
    auto ordering_info = substitute(
        ^^std::type_order_v, {
            substitute(^^_Helper, {std::meta::reflect_constant(a)}),
            substitute(^^_Helper, {std::meta::reflect_constant(b)})
        }
    );
    return extract<const std::strong_ordering&>(ordering_info);
} else {
    // non-types compare less than types
    return is_exactly_type(a) <=> is_exactly_type(b);
}
}
```

I have no implementation of the same comparison for built-in `operator<=>` in a compiler.

§ 6. Wording

Wording is relative to [N5032].

§ 6.1. [basic.fundamental]

...

- 19 There is an *implementation-defined strict total order of reflections*, such that reflection values that compare unequal (`expr.eq`) are also inequivalent in this total order.

- [Note 1: `^^int`, `^^const_int`, and `^^int` compare unequal to each other. - end note]
- [Note 2: This ordering need not be consistent with the one introduced by `type_info::before`, when applied to reflections that represent types. - end note]
- [Note 3: The ordering of reflections that represent TU-local entities from different translation units is not observable, because it is impossible to form corresponding relational or three-way comparison expressions. - end note]

- 20 *Recommended practice*: The order should be lexicographical on parameter-type-lists and template argument lists when applied to reflections representing functions and template specializations.

§ 6.2. [expr.spaceship]

...

- 7 If both operands have type `std::meta::info`, the result type is `std::strong_ordering`. The result is `std::strong_ordering::less` if the first operand precedes the second operand, `std::strong_ordering::equal` if the operands compare equal, or `std::strong_ordering::greater` if the second operand precedes the first operand according to the implementation-defined strict total order of reflections ([basic.fundamental]).

6.3. [expr.rel]

...

- 2 The converted operands shall have arithmetic, enumeration, ~~or~~ pointer `,or std::meta::info` type.

...

- 7 If both operands (after conversions) have type `std::meta::info`, each of the operators shall yield `true` if the specified relationship is true corresponding to the implementation-defined strict total order of reflections ([basic.fundamental]), otherwise `false`.

6.4. [over.built]

...

- 15 For every T, where T is an enumeration type ~~or~~ a pointer type `,or std::meta::info`, there exist candidate operator functions of the form

```
bool operator==(T, T);
bool operator!=(T, T);
bool operator<(T, T);
bool operator>(T, T);
bool operator<=(T, T);
bool operator>=(T, T);
R operator<=>(T, T);
```

where R is the result type specified in [expr.spaceship].

- 16 For every T, where T is a pointer-to-member type ~~,or std::meta::info~~, or `std::nullptr_t`, there exist candidate operator functions of the form

```
bool operator==(T, T);
bool operator!=(T, T);
```

6.5. [compare.type]

- 1 ~~There is an implementation defined total ordering of all types. For any (possibly incomplete) types X and Y, the expression `TYPE_ORDER(X, Y)` is a constant expression ([expr.const]) of type `strong_ordering` ([cmp.strongord]). Its value is `strong_ordering::less` if X precedes Y in this implementation-defined total order, `strong_ordering::greater` if Y precedes X, and `strong_ordering::equal` if they are the same type.~~

~~— [Note 1: `int`, `const int` and `int&` are different types. — end note]~~

~~— [Note 2: This ordering need not be consistent with the one induced by `type_info::before`. — end note]~~

~~— [Note 3: The ordering of TU-local types from different translation units is not observable, because the necessary specialization of `type_order` is impossible to name. — end note]~~

```
template<class T, class U> struct type_order {
    static constexpr strong_ordering value = TYPE_ORDER(T, U)^T <=> ^U;

    using value_type = strong_ordering;

    constexpr operator value_type() const noexcept { return value; }
    constexpr value_type operator()() const noexcept { return value; }
};
```

...

- 3 ~~Recommended practice: The order should be lexicographical on parameter type lists and template argument lists~~

6.6. [cpp.predefined]

```
#define __cpp_meta_info_order DATE-OF-ADOPTION
```

References

Informative References

[N5032]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 15 December 2025. URL: <https://wg21.link/n5032>

[P2830R10]

Gašper Ažman, Nathan Nichols. *Standardized Constexpr Type Ordering*. 15 March 2025. URL: <https://wg21.link/p2830r10>

[P2996R13]

Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. *Reflection for C++26*. 20 June 2025. URL: <https://wg21.link/p2996r13>